California State University Long Beach

Computer Engineering and Computer Science Department

CECS 440 – Computer Architecture

Lab 7 – Write Back

Objectives:

- Implement the **Write Back (WB) stage** of a single-cycle MIPS processor in Verilog.
- Use control signals (**RegWriteM**, **MemtoRegW**) to manage data flow to the register file.
- Connect and verify data paths for **ALUOutW**, **ReadData**, and **WriteRegM**.
- Simulate and confirm correct register updates based on control signal using **Xilinx Vivado** behavioral simulation with a Verilog testbench.

Background:

In the single-cycle MIPS processor, instruction execution occurs across five stages. The simplest of these is the **Write Back (WB) stage**, which primarily consists of a single multiplexer and several buses (collections of wires used to transport multiple bits).

During this stage, data is written back to the register file based on the control signals and data values propagated from previous stages. The key components and signals involved in the WB stage are described below:

- The **Register Write (RegWriteW)** control signal is passed from the **MEM** stage to the **WB** stage. It determines whether the result should be written into a register (inside the Register File).
    - The **WriteRegW** carries the destination register address to identify which register in the register file should be updated.
- The **MemtoRegW** control signal specifies the source of the data to be written back — either the result of the **ALU operation (ALU output)** or the value read from **data memory (ReadData)**.
    - The **ALUOutW** bus carries the 32-bit output result from the ALU computation performed in earlier stages.
    - The **Read DataW** bus carries the 32-bit value retrieved from **DMEM**.

In summary, the WB stage finalizes the instruction's execution by updating the register file with the appropriate data, based on the control signals and data paths defined above.

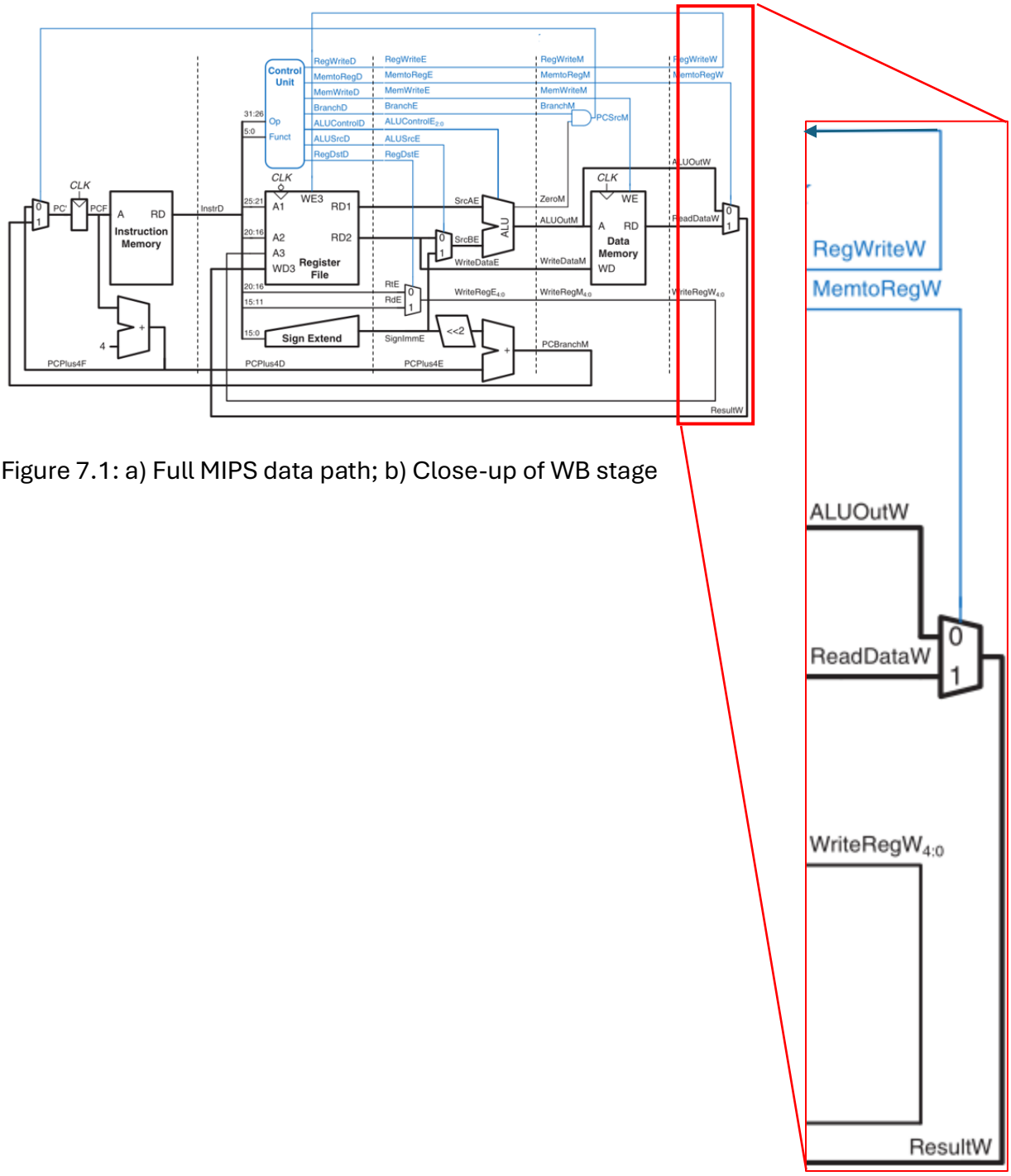

Figure 7.1: a) Full MIPS data path; b) Close-up of WB stage

Steps:

1. A skeleton code for a parameterized multiplexer (mux) is provided for use in this and other designs, as illustrated in Figure 7.2. Complete the missing portion of the code.

```
1  `timescale 1ns/1ps
2
3  module mux#(parameter WIDTH = 8)
4    (input [WIDTH-1:0] d0, input [WIDTH-1:0] d1, input s, output [WIDTH-1:0] y);
5
6    assign y = ;
7
8  endmodule
```

Figure 7.2: Skeleton Code for 2-1 Mux

2. A skeleton code for Write Back stage is provided in Figure 7.3. Complete the missing portion of the code.

```
1  // Code your design here
2  `timescale 1ns/1ps
3
4  `include "mux.v"
5
6  module WB_Stage(
7    input regwritem, memtoregw,
8    input [31:0] aluoutw, readdata,
9    input [4:0] writeregm,
10   output      regwritew,
11   output [4:0] writeregw,
12   output [31:0] resultw);
13
14   assign regwritew = regwritem,
15   writeregw = writeregm
16
17   // mux
18   mux #(32) resmux();
19
20 endmodule
```

Figure 7.3: Skeleton Code for Write Back stage

3. Finally, test your design to ensure it works correctly using the Verilog test fixture code in Figure 7.4. Correct test results are shown in Figure 7.5.

```
`timescale 1ns/1ps
module t_WB_Stage();
  reg regwritem, memtoregm;
  reg [31:0] aluoutw, readdata;
  reg [4:0] writeregm;
  wire regwritew;
  wire [4:0] writeregw;
  wire [31:0] resultw;
  integer i;

  WB_Stage dut (regwritem, memtoregm, aluoutw,readdata,
                writeregm,regwritew,writeregw, resultw);

  initial begin
    for (i = 0; i<5; i = i + 1) begin
      aluoutw = $random;
      readdata = $random;
      {writeregm,regwritem} = $random;
      memtoregm = i;
      #1 testcase();
    end
    #1 $finish;
  end

  task testcase();begin
    $display ("Test Case %d",i);
    $display ("aluoutw =  %h\t  readdata = %h\t", aluoutw, readdata);
    $display ("memtoregm =  %h\t  resultw = %h\t", memtoregm, resultw);

    if((!memtoregm && resultw != aluoutw) ||
       (memtoregm && resultw != readdata))
      begin
        $display("TEST FAILED: resmux has malfunctioned");
        $finish;
      end

    $display("regwritem = %b\t regwritew = %b",regwritem,regwritew);
    $display("writeregm = %h\t writeregw = %h", writeregm,writeregw);

    if((regwritem!=regwritew)||(writeregm!=writeregw)) begin
      $display("TEST FAILED: signal passing malfunctioned");
      $finish;
    end
    $display("");
  end endtask
endmodule
```

Figure 7.4 Verilog code for testbench

```
Test Case           0
aluoutw = 12153524      readdata = c0895e81
memtoregm = 0   resultw = 12153524
regwritem = 1   regwritew = 1
writeregm = 04  writeregw = 04

Test Case           1
aluoutw = b1f05663      readdata = 06b97b0d
memtoregm = 1   resultw = 06b97b0d
regwritem = 1   regwritew = 1
writeregm = 06  writeregw = 06

Test Case           2
aluoutw = b2c28465      readdata = 89375212
memtoregm = 0   resultw = b2c28465
regwritem = 1   regwritew = 1
writeregm = 00  writeregw = 00

Test Case           3
aluoutw = 06d7cd0d      readdata = 3b23f176
memtoregm = 1   resultw = 3b23f176
regwritem = 1   regwritew = 1
writeregm = 1e  writeregw = 1e

Test Case           4
aluoutw = 76d457ed      readdata = 462df78c
memtoregm = 0   resultw = 76d457ed
regwritem = 1   regwritew = 1
writeregm = 1c  writeregw = 1c
```

Figure 7.5: Expected output

Submission:

Once you have verified that your project is functioning correctly, copy the contents of **design.v, mux.v** and **design_tb.v** into a single text file named **lab7.txt**. Submit **this file** along with your **lab report** on Canvas.

Write your report using the **Lab Report Template** available on Canvas. Be sure to include:

- Screenshots of your code
- Console output
- One waveform screenshot showing a few selected test cases

Note: Keep all lab files, as they will be needed for future labs.